Simple RISC

Simple RISC

Since all instructions are 4-byte aligned, their addresses always end in `00`; therefore, these two bits are not stored and are implicitly assumed, allowing a larger effective branch range.

so all locations will be correctly reached and no info is lost

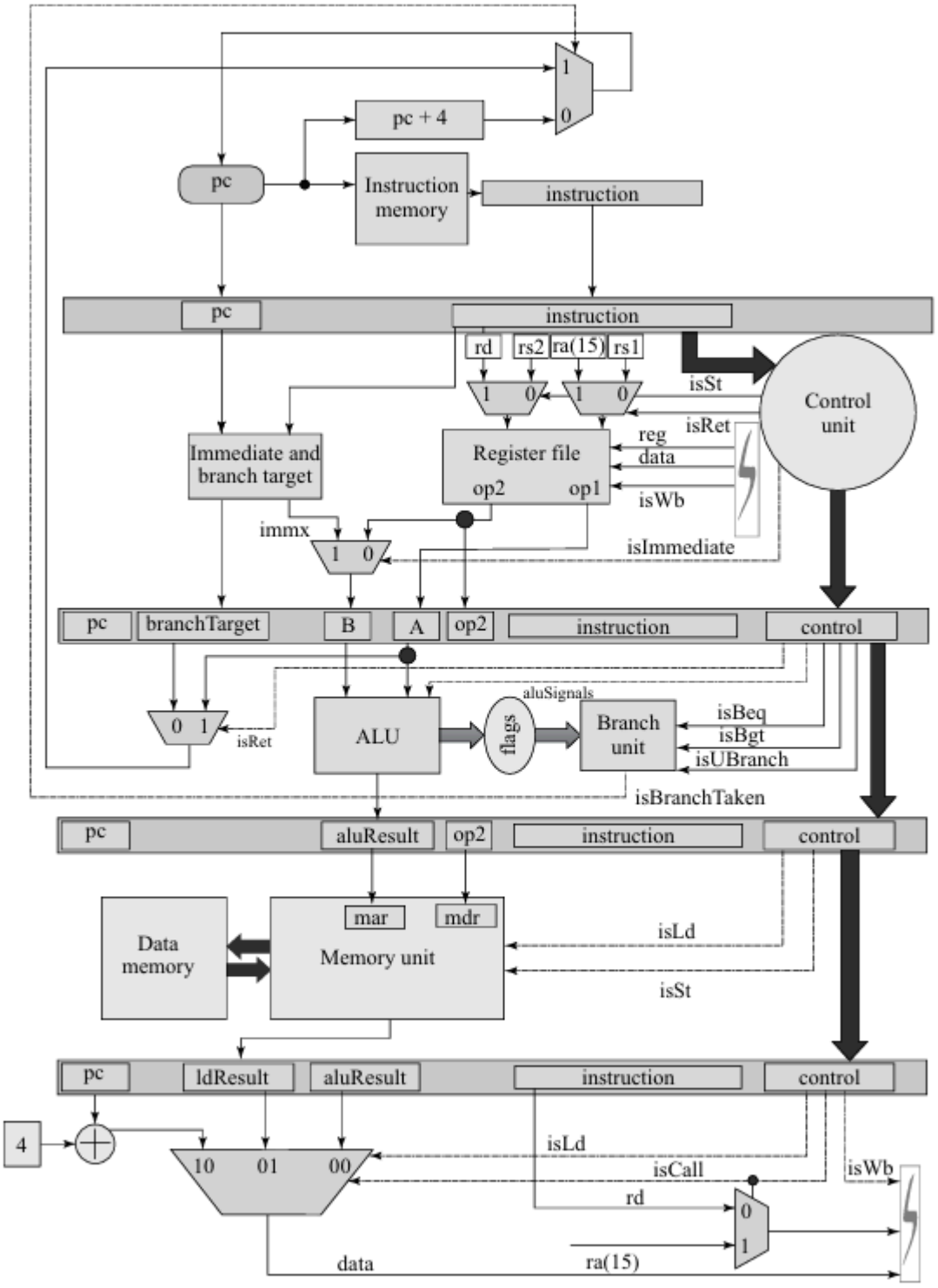


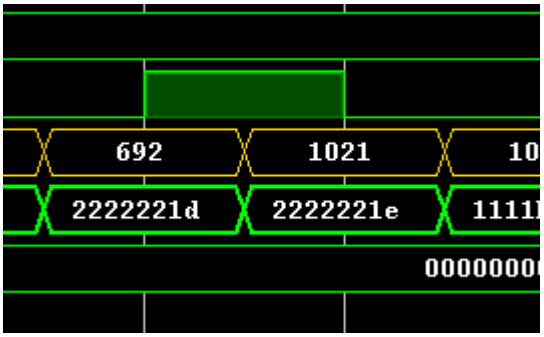
Fig. 9.9 Pipelined data path

Instruction fetch unit

1. Mux input is directly handling the pc incrementing task so no external incrementing unit is used.
2. PC is a 10 bit register which points to each 32 bit instruction from 0 to 1024 in decimal

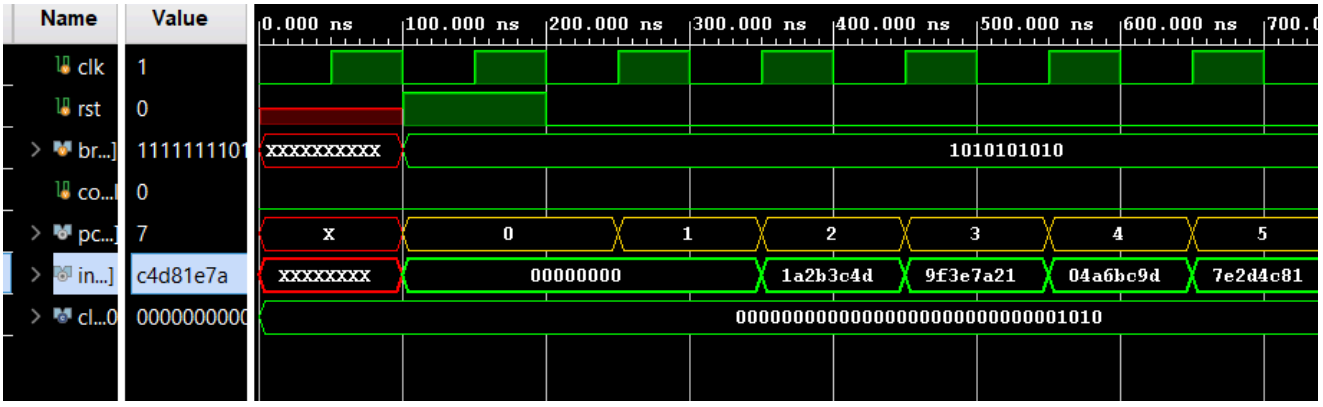
3. PC is incremented as 0 ,1, 2, 3 etc

issue




```
690 2222221B
691 2222221C
692 2222221D
693 2222221E
694 2222221F
695 22222220
696 33333310
697 33333311
```

since the data encoding in the file starts from the index 1 and ends at 1024 but the processor is designed in a way that it is from 0 to 1023 in the pc.



The image shows the issue clearly since the PC [0] is clearly meaning that the first instruction must be taken but there is no instruction in the first section because the mem file encodes the hex code starting from the index 1 and not 0.

```
1 00000000
2 1A2B3C4D
3 9F3E7A21
4 04A6BC9D
```

HENCE,

There is a slight mismatch in the instruction picked by the processor when a branch is taken.

The branch is not relative addressing. When the branch is taken it just go to the location by loading the PC with the new address

```
    if(reset_p) // reset the program counter
        begin
            out_PC<=10'd1; // Make this count zero when doing synthesize because
            // In actual hardware the count actually starts from the value zero not 1
        end

    else
```

IN the PC module i have changed the count to start from this

OR

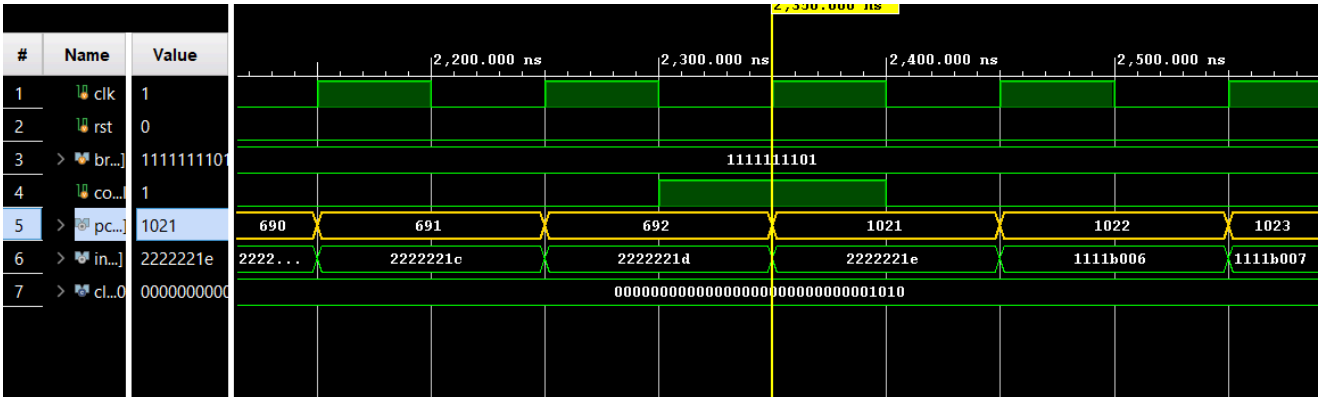
May be its just doing the required it just points to the next instruction to be fetched.

Also

IMP NOTE

IF you want to jump to location put the address of the location prior to that ,
because the location to which is supposed to jump,
It wont pick the instruction at that address .
At that PC count it shows the very next instruction from the point it jumped.

ROM is made unclocked now its not showing 693th data, at 1021 now its showing the 1022 th data at 1021



It is showing correctly from the address 1022

`this may not be an issue because the memory unit only fetches the data when the clock pulse arrives and the PC is also incremented at the same clock pulse so the memory location it address also changes.

This below note is from chat gpt

On a real FPGA (Vivado / BRAM reality)

FPGA block RAMs are:

- **Synchronous read**
- Output valid **1 clock after address**

Meaning:

```
always @(posedge clk) instruction <= IMEM[PC];
```

✓ Maps cleanly to BRAM
✓ Fast, scalable
❌ Adds **1-cycle latency**

Vivado strongly prefers this.

What YOU should use (for your design)
Use **clocked instruction memory**

Because:

- You are on FPGA
- You already wrote it that way
- Pipelined CPUs naturally absorb the latency

Your current `instruction_mem` is **correct and realistic**.

How this fits into the pipeline (VERY important)

With synchronous instruction memory:

Cycle	PC	IMEM output
N	PC = X	—
N+1	PC = X+1	instruction @ X

So your pipeline becomes:

```
IF stage: Cycle N: PC update Cycle N+1: instruction available
```

Which means:
👉 You **must** have an **IF/ID register**.

ID stage

The reg bank reading is combinational
The return address is always 15 in the simple risc ,

But in this we can choose what its value is

Control

`flags

Greater than
Equal
Zero
Carry
Overflow

carry in for ALU
User defined 1

ASC module

task Sub_num;
input [31:0]A;
input [31:0]B;

```
          begin
              Areg=A;
              Breg=((~B)+1'b1);

          end
        endtask
```

1. in the subtraction part there is no input carry used
2. No overflow
3. Comparison not taken into account about the sign
4. Cout flag always considered rather than only in Additon
5. Multiplier and Divider need to be improved

CLEAR INSTRUCTION IS NEEDED SINCE THE FLAGS WONT GET AUTO CLEARED
BIT ADDRESSIBLE MODE

The register has a write signal which must be made high inorder to write the data into the register bank --> COnnect it to the isWb signal form the last stage (i hope i am correct)

REG BANK

The situation in which the processor trying to read and write to the same location is not handled for reg bank as well as any memory element in the design.

RAM address

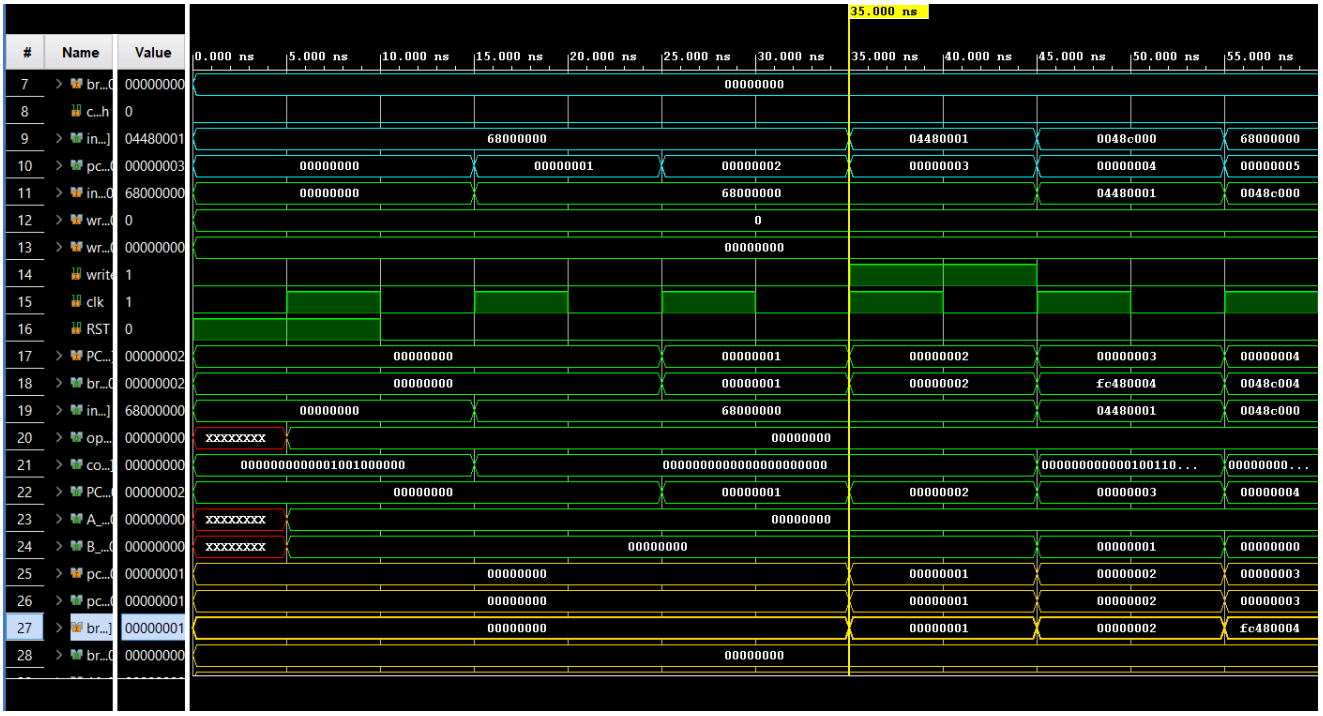
The RAM contains only 1024 locations but the MAR gets a 32 bit value from the aluresult.
The RAM that is addressed by aluresult is only using 10 bits for adress
Data width is 32 bit for each RAM locations.

Rest location in the MAR (22 bits free) can be used later for future expansion or anything else

CARRY out signal deleted from the top module

TEST with instructions (one by one)

Nop --> 68000000
ADD immediate -- > opCode = 04480001 (R1<--R2+imm [1])
ADD register --> opCode = 0048c000 (R1<--R2+R3)



Branch instruction

Branch instructino (b) [90000006] (+ branching) (PC+6)
Should it stop other instructions from fetching?

When branch is excecuted the next two instructions after the branch is also getting into the pipeline.

negative branching instruction
9FFFFFFE sucessfully done

68000000
68000000
68000000
04480001
0048c000

68000000
68000000
90000006
68000000
68000000
68000000
68000000
68000000
04480001
0048c000
68000000
68000000
9FFFFFFE
04480001
0048c000
(

)

Again the PC anomaly .....

CMP instruction

use RD 0000

CMP 2804

bgt

68000000
68000000
68000000
04480001
68000000
68000000
68000000
68000000
68000000
68000000
28048000 r1 and r2 comparison
04480001
68000000

8FFFFFFE
68000000
68000000
68000000
68000000
68000000

beq

r3 and r4 comparison
280D0000

MOV instruction
mov r7<--
68000000
68000000
68000000
4DC00003
68000000
68000000
68000000
68000000
68000000

call

68000000
68000000
68000000
4DC00003
68000000
68000000
68000000
68000000
68000000
9800000A
68000000
68000000
4DC00003
68000000
68000000
68000000
68000000
68000000
68000000

68000000
4DC00004
68000000
68000000
68000000
A0000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000

7C80000F
0111 1100 1000 0000 0000 0000 0000 1111

74C0000F
0111 0100 1100 0000 0000 0000 0000 1111

LOAD AND STORE
68000000
4C800007
68000000
68000000
68000000
68000000
68000000
68000000
68000000
7C80000F
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
74C0000F

68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000
68000000

Multiplication

0100 1110 1000 0000 0000 0000 0000 0101
4E80005

68000000
68000000
68000000
68000000
68000000
68000000
68000000
4E800005
68000000
68000000
68000000
68000000
7E8003FF
68000000
68000000
68000000
68000000
68000000
748003FF
68000000
68000000
68000000
68000000
68000000
54C80002
55080003
68000000
68000000
68000000

68000000
68000000
68000000
014D0000
68000000
68000000
68000000
68000000
68000000
68000000